

A Technical Introduction to DBSCAN

Section 1

History In 1972, Robert F. Ling published a closely related algorithm in "The Theory and Construction of k-Clusters" in The Computer Journal with an estimated runtime complexity of $O(n^3)$. DBSCAN has a worst-case of $O(n^2)$, and the database-oriented range-query formulation of DBSCAN allows for index acceleration. The algorithms slightly differ in their handling of border points.

Section 2

Parameter estimation Every data mining task has the problem of parameters. Every parameter influences the algorithm in specific ways. For DBSCAN, the parameters ϵ and minPts are needed. The parameters must be specified by the user. Ideally, the value of ϵ is given by the problem to solve (e.g. a physical distance), and minPts is then the desired minimum cluster size.

Section 3

$$\min_{C \subset C} \max_{p, q \in C} d(p, q) \leq \epsilon \quad \forall p, q \in C \quad \forall C \in \mathcal{C} \mid C \subseteq \{C_1, \dots, C_n\} \text{ and } \forall p, q \in C, d(p, q) \leq \epsilon$$

Section 4

Optimization Criterion DBSCAN optimizes the following loss function: For any possible clustering $C = \{C_1, \dots, C_n\}$ out of the set of all clusterings $\mathcal{C}$, it minimizes the number of clusters under the condition that every pair of points in a cluster is density-reachable, which corresponds to the original two properties "maximality" and "connectivity" of a cluster:

Section 5

Relationship to spectral clustering A spectral implementation of DBSCAN is related to spectral clustering in the trivial case of determining connected graph components — the optimal clusters with no edges cut. However, it can be computationally intensive, up to $O(n^3)$. Additionally, one has to choose the number of eigenvectors to compute. For performance reasons, the original DBSCAN algorithm remains preferable to its spectral implementation.

Section 6

Find the points in the ϵ (eps) neighborhood of every point, and identify the core points with more than minPts neighbors. Find the connected components of core points on the neighbor graph, ignoring all non-core points. Assign each non-core point to a nearby cluster if the cluster is an ϵ (eps) neighbor, otherwise assign it to noise. A naive

implementation of this requires storing the neighborhoods in step 1, thus requiring substantial memory. The original DBSCAN algorithm does not require this by performing these steps for one point at a time.

Section 7

Apache Commons Math contains a Java implementation of the algorithm running in quadratic time. ELKI offers an implementation of DBSCAN as well as GDBSCAN and other variants. This implementation can use various index structures for sub-quadratic runtime and supports arbitrary distance functions and arbitrary data types, but it may be outperformed by low-level optimized (and specialized) implementations on small data sets. mlpack includes an implementation of DBSCAN accelerated with dual-tree range search techniques. PostGIS includes ST_ClusterDBSCAN – a 2D implementation of DBSCAN that uses R-tree index. Any geometry type is supported, e.g. Point, LineString, Polygon, etc. R contains implementations of DBSCAN in the packages dbscan and fpc. Both packages support arbitrary distance functions via distance matrices. The package fpc does not have index support (and thus has quadratic runtime and memory complexity) and is rather slow due to the R interpreter. The package dbscan provides a fast C++ implementation using k-d trees (for Euclidean distance only) and also includes implementations of DBSCAN*, HDBSCAN*, OPTICS, OPTICSXi, and other related methods. scikit-learn includes a Python implementation of DBSCAN for arbitrary Minkowski metrics, which can be accelerated using k-d trees and ball trees but which uses worst-case quadratic memory. A contribution to scikit-learn provides an implementation of the HDBSCAN* algorithm. pyclustering library includes a Python and C++ implementation of DBSCAN for Euclidean distance only as well as OPTICS algorithm. SPMF includes an implementation of the DBSCAN algorithm with k-d tree support for Euclidean distance only. Weka contains (as an optional package in latest versions) a basic implementation of DBSCAN that runs in quadratic time and linear memory. linfa includes an implementation of the DBSCAN for the rust programming language. Julia includes an implementation of DBSCAN in the Julia Statistics's Clustering.jl package.

Section 8

Disadvantages DBSCAN is not entirely deterministic: border points that are reachable from more than one cluster can be part of either cluster, depending on the order the data are processed. For most data sets and domains, this situation does not arise often and has little impact on the clustering result: both on core points and noise points, DBSCAN is deterministic. DBSCAN* is a variation that treats border points as noise, and this way achieves a fully deterministic result as well as a more consistent statistical interpretation of density-connected components. The quality of DBSCAN depends on the distance measure used

in the function $\text{regionQuery}(P, \epsilon)$. The most common distance metric used is Euclidean distance. Especially for high-dimensional data, this metric can be rendered almost useless due to the so-called "Curse of dimensionality", making it difficult to find an appropriate value for ϵ . This effect, however, is also present in any other algorithm based on Euclidean distance. DBSCAN cannot cluster data sets well with large differences in densities, since the minPts - ϵ combination cannot then be chosen appropriately for all clusters. If the data and scale are not well understood, choosing a meaningful distance threshold ϵ can be difficult. See the section below on extensions for algorithmic modifications to handle these issues.

Section 9

Advantages DBSCAN does not require one to specify the number of clusters in the data a priori, as opposed to k-means. DBSCAN can find arbitrarily-shaped clusters. It can even find a cluster completely surrounded by (but not connected to) a different cluster. Due to the MinPts parameter, the so-called single-link effect (different clusters being connected by a thin line of points) is reduced. DBSCAN has a notion of noise, and is robust to outliers. DBSCAN requires just two parameters and is mostly insensitive to the ordering of the points in the database. (However, points sitting on the edge of two different clusters might swap cluster membership if the ordering of the points is changed, and the cluster assignment is unique only up to isomorphism.) DBSCAN is designed for use with databases that can accelerate region queries, e.g. using an R^* tree. The parameters minPts and ϵ can be set by a domain expert, if the data is well understood.